Utilidad para dibujo del Dendograma

```
import numpy as np

from matplotlib import pyplot as plt
from scipy.cluster.hierarchy import dendrogram
from sklearn.datasets import load_iris
from sklearn.cluster import AgglomerativeClustering
from scipy.cluster.hierarchy import set_link_color_palette
```

Print to PDF

Función de dibujo de Elipses (Clustering Mezclas Gaussianas)

```
import matplotlib as mpl
def make_ellipses(gmm, ax):
    for n, color in enumerate(colors):
        if gmm.covariance_type == "full":
            covariances = gmm.covariances_[n][2:, 2:]
        elif gmm.covariance_type == "tied":
            covariances = gmm.covariances_[2:, 2:]
        elif gmm.covariance_type == "diag":
            covariances = np.diag(gmm.covariances_[n][2:])
        elif gmm.covariance_type == "spherical":
            covariances = np.eye(gmm.means_.shape[1]) *
gmm.covariances_[n]
        v, w = np.linalg.eigh(covariances)
        u = w[0] / np.linalg.norm(w[0])
        angle = np.arctan2(u[1], u[0])
        angle = 180 * angle / np.pi  # convert to degrees
        v = 2.0 * np.sqrt(2.0) * np.sqrt(v)
        ell = mpl.patches.Ellipse(
            gmm.means_[n, 2:], v[0], v[1], 180 + angle, color=color
        )
        ell.set_clip_box(ax.bbox)
        ell.set_alpha(0.5)
        ax.add_artist(ell)
        ax.set_aspect("equal", "datalim")
```

Función a la medida para graficar el Dendograma

Utiliza la paleta de colores básica de pyplot:

['b', 'g', 'r', 'c', 'm', 'y'] : Azul, Verde, Rojo, Cian, Magenta, Amarillo

En la llamada a la función se reserva el color 'k' (Negro) para los puntos aislados en el dendograma. Esto se realiza con la variable above_threshold_color.

Para mejorar la visión se ajusta el tamaño del fuente (leaf_font_size) a 10

La función **scipy.cluster.hierarchy.dendrogram** a la que se llama en **plot_dendrogram** tiene un parámetro opcional que es **color_threshold**. Este parámetro le sirve para situar la linea de corte de grupos en el dendograma. Por debajo de esta distancia crea grupos con un color distinto cada uno, salvo que estén aislados.

Por defecto, si no se pasa ningún valor, toma **0.7 * Altura_maxima** . Se incorpara código a **plot_programa** para calcular automáticamente la altura umbral en función de los grupos formados en el cluster

Se puede pasar un bloque de parámetros propios de la función scipy.cluster.hierarchy.dendrogram (en **kwargs**):

- Para mejorar la visión se ajusta el tamaño del fuente en **leaf_font_size** (por ejemplo a 10)
- Todos los parámetros disponibles en: https://docs.scipy.org/doc/scipy/reference/generated/scipy.cluster.hierarchy.dendrogram.html

```
import numpy as np

from matplotlib import pyplot as plt
from scipy.cluster.hierarchy import dendrogram
from sklearn.datasets import load_iris
from sklearn.cluster import AgglomerativeClustering
from scipy.cluster.hierarchy import set_link_color_palette

def plot_dendrogram(model, titulo, subtitulo, plotSize, **kwargs):
    # Create linkage matrix and then plot the dendrogram
    # plotSize=(15, 10)

    ##Busqueda automática de la distancia umbral
    nSamples = len(model.labels_)
    maxChild =
max(max(model.children_[:,0]),max(model.children_[:,1]))+1
    etiquetaNodo=[-1 for i in range(maxChild)]
    distBajoUmbral=[]
    for i, nodo in enumerate(model.children_):
        etiq1 = model.labels_[nodo[0]] if nodo[0] < nSamples else
etiquetaNodo[nodo[0]]
        etiq2 = model.labels_[nodo[1]] if nodo[1] < nSamples else
etiquetaNodo[nodo[1]]
        if etiq1 == etiq2 and etiq1>-1:
            etiquetaNodo[i+nSamples]=etiq1
            distBajoUmbral.append(model.distances_[i])
    distanciaUmbral = 0 if distBajoUmbral==[] else max(distBajoUmbral)
    if distanciaUmbral > 0:  ## La distancia umbral ha de queda en punto
medio valor calculado más distancia siguiente nodo
        distanciaUmbral = 0.5*(distanciaUmbral +
min(model.distances_[model.distances_ > distanciaUmbral]))
    if 'color_threshold' not in  kwargs.keys():  ## Si el valor no viene
por parámetro se añade
        kwargs['color_threshold'] = distanciaUmbral


    # create the counts of samples under each node
    counts = np.zeros(model.children_.shape[0])
    n_samples = len(model.labels_)
    for i, merge in enumerate(model.children_):
        current_count = 0
        for child_idx in merge:
            if child_idx < n_samples:
                current_count += 1  # leaf node
            else:
                current_count += counts[child_idx - n_samples]
        counts[i] = current_count

    linkage_matrix = np.column_stack([model.children_, model.distances_,
                                      counts]).astype(float)
    # Plot the corresponding dendrogram
    #dendrogram(linkage_matrix, **kwargs)

    # Plot the corresponding dendrogram
    fig, ax = plt.subplots(1, 1, figsize=plotSize)  # set size
    fig.suptitle(titulo)
    set_link_color_palette(['b', 'g', 'r', 'c', 'm', 'y'])
    if 'above_threshold_color' not in  kwargs.keys():  ## Si el valor no
viene por parámetro se añade
        kwargs['above_threshold_color'] = 'k'
    ax = dendrogram(linkage_matrix, **kwargs)


    plt.tick_params(axis='x', bottom='off', top='off',
labelbottom='off')
    plt.tight_layout()
    plt.xlabel(subtitulo)

    ### Se añade una barra con el umbral
    w_c = kwargs['above_threshold_color']
    #w_y = 0.7*model.distances_[-1] if 'color_threshold' not in
kwargs.keys() else kwargs['color_threshold']
    w_y = kwargs['color_threshold']
    if w_y > 0:
        xmin, xmax, ymin, ymax = plt.axis()
        plt.plot([w_y for n in range(int(xmax-xmin))], c=w_c, ls='--')

    plt.show()
```

INDICE VI (VARIACIÓN DE LA INFORMACIÓN) DE MARINA

MEILĂ

```
import numpy as np
from sklearn.metrics.cluster import mutual_info_score
def EntropiaCluster(y):
    n = np.size(y)
    etiquetas = np.unique(y)
    entropia=0
    for et in etiquetas:
        fr = np.size(y[y==et])/n
        entropia += -fr*np.log(fr)
    return entropia
def probEtiqueta(y, et):
    return np.size(y[y==et])/np.size(y)
def probConjunta(y1, y2, et1, et2):
    ## Se parte que el nº de elementos en y1 e y2 es el mismo
    return np.size(y1[(y1==et1)*(y2==et2)])/np.size(y1)
def informacionMutua(y1, y2):
    etiquetas1 = np.unique(y1)
    etiquetas2 = np.unique(y2)

    infoMutua=0
    for et1 in etiquetas1:
        for et2 in etiquetas2:
            if probConjunta(y1, y2, et1, et2)>0:
                infoMutua += probConjunta(y1, y2, et1,
et2)*np.log(probConjunta(y1, y2, et1, et2)/(probEtiqueta(y1,
et1)*probEtiqueta(y2, et2)))
    return infoMutua
def variationInformation(y1, y2):
    #return EntropiaCluster(y1) + EntropiaCluster(y2) -
2*informacionMutua(y1, y2)
    return EntropiaCluster(y1) + EntropiaCluster(y2) -
2*mutual_info_score(y1, y2)

def adjustedVariationInformation(y1, y2):
    return variationInformation(y1, y2) / np.log(np.size(y1))

def normalizedVariationInformation(y1, y2):
    return variationInformation(y1, y2) / (EntropiaCluster(y1) +
EntropiaCluster(y2))
```

```
print('load done!')
```

```
load done!
```

Previous
Capítulo 3: Clustering

Next
03.1 Aprendizaje no supervisado. Clustering

By Departamento de Matemática Aplicada
© Copyright 2021.